



OPENGL

Instructor: Dinesh Maharjan

2022



OpenGL

- OpenGL (Open Graphics Library)
- Developed by SGI (Silicon Graphics Incorporated) in 1992
- OpenGL includes many functions that can be used to render high quality 2D and 3D scenes.
- It includes function to generate primitives such as lines, points, polygons.
- It can be used to control light, transparency, surface rendering.
- It is built on Graphics hardware.
- So, It is very efficient.
- It is platform independent, display device independent.



OpenGL

- OpenGL is window system independent.
- It does not include any window related functions such as create window, resize window, reposition window, event handling.
- It needs other APIs to perform window related tasks.
- X Window System: GLX
- Apple Machintosh: AGL
- Windows System: WGL
- These libraries can be used to create GUIs such as slider, button, menus.



OpenGL Functions

- OpenGL has mainly two types of function.
-
- 1. Primitive Generating Functions
- 2. State Changing Functions.



Primitive Generating Functions

- These functions can be used to generate primitives such as line, point, triangle, and other types of polygon.
- Basically, these functions are kept inside our own callback function.
- So, we need to define a callback function which will create the scene in frame buffer and render it on screen.
- This function is called by `glutDisplayFunc()`

Primitive Generating Functions

- `void draw(){`
 - `glPointSize(4);`
 - `glBegin(GL_POINTS);`
 - `glVertex2f(0.2,0.4);`
 - `glVertex2f(0.2,0.6);`
 - `glEnd();`
 - `glFlush();`
 - `}`



Primitive Generating Functions

- Here, void draw is a callback function that we need to define in every OpenGL program.
- glBegin() and glEnd() will include vertices of the object to be rendered.
- The parameter of glBegin() can be one of the following.

Primitive Types



All geometric primitives are specified by vertices



GL_POINTS



GL_LINES



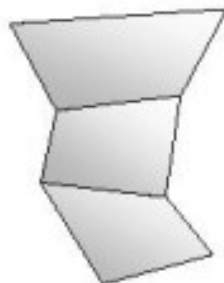
GL_LINE_STRIP



GL_LINE_LOOP



GL_POLYGON



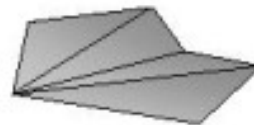
GL_QUAD_STRIP



GL_QUADS



GL_TRIANGLE_STRIP

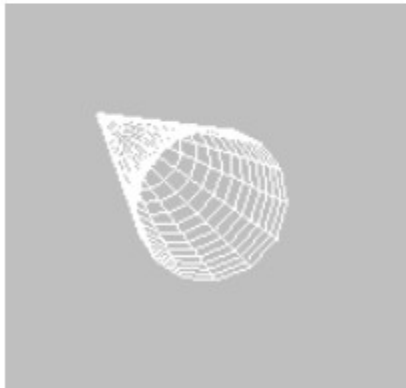


GL_TRIANGLE_FAN



GL_TRIANGLES

```
glBegin(GL_LINES) ;  
    [lots of  
    glVertex  
    calls] ;  
glEnd() ;
```

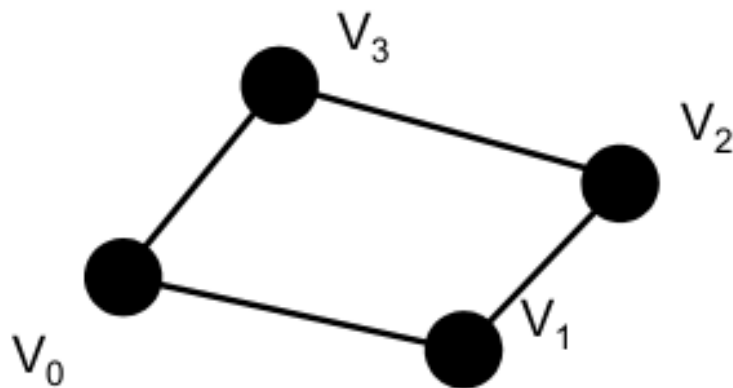


```
glBegin(GL_QUADS) ;  
    [lots of  
    glVertex  
    calls] ;  
glEnd() ;
```



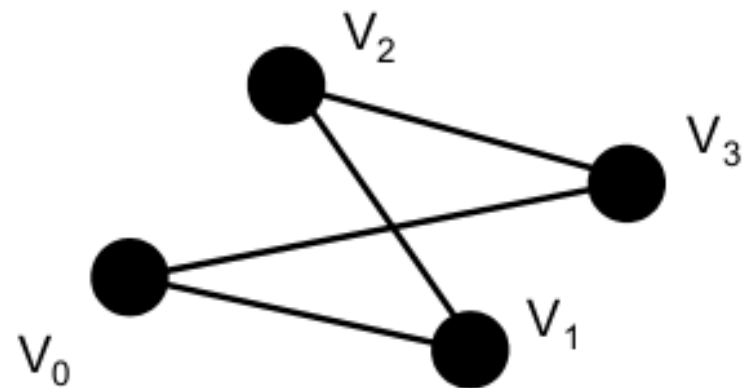
OpenGL Topologies – Vertex Order Matters

GL_LINE_LOOP



Probably what you meant to do

GL_LINE_LOOP



Probably not what you meant to do

`glVertex2f(x, y)`

number of
Components/
Dimensions

2 – (x,y)

3 – (x,y,z)

4 – (x,y,z,w) or
(r,g,b,a)

B – byte

ub – unsigned byte

s – short

us – unsigned short

i – int

ui – unsigned int

f – float

d – double

Add 'v' for vector
form

`glVertex2fv(v)`



Primitive Generating Functions

- - **glFlush()** will send the scene in frame buffer to the screen.
- And, scene gets rendered.



State Changing Functions

- OpenGL is also known as the state machine.
- It maintains modelview matrix, projection matrix, drawing color, background color, size of points and width of line etc.
- These state remains unchanged until we change them.
- So, some of important state changing functions are `glClearColor()`, `glColor3f()`, `glClear()`, `glPointSize()`, `glLineWidth()`

State Changing Functions

- **glClearColor(<red>,<green>,<blue>,<opacity>);**
- This defines the background color.
- It will not set the background color until we call glClear() function.
- Value of all parameters can be in [0,1].
- For defining red background, glClearColor(1,0,0,1);
- For white, glClearColor(1,1,1,1);
- The opacity defines transparency when two objects overlap.
- 0 means less opaque and 1 means high opaque
- Black is default background color.



State Changing Functions

- **glClear(GL_COLOR_BUFFER_BIT);**
- This will clear the contents of frame buffer and then sets the background color that is defined by `glClearColor()`
- `GL_COLOR_BUFFER_BIT` refers to frame buffer.

State Changing Functions

- **glColor3f(<red>,<green>,<blue>);**
- This defines the drawing color.
- This is done before glBegin();
- The value of red, green, and blue is in [0,1]
- For setting green drawing color,
glColor3f(0,1,0)
- For White, glColor(1,1,1);
- White is default drawing color.

`glColor3f(0.0, 0.0, 0.0)` – black

`glColor3f(1.0, 0.0, 0.0)` – red

`glColor3f(0.0, 1.0, 0.0)` – green

`glColor3f(0.0, 0.0, 1.0)` – blue

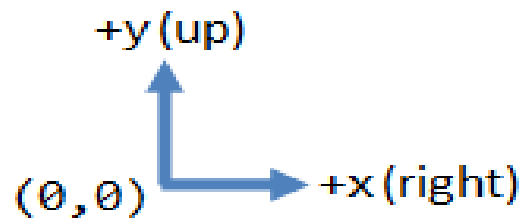
`glColor3f(1.0, 1.0, 0.0)` – yellow

`glColor3f(1.0, 0.0, 1.0)` – magenta

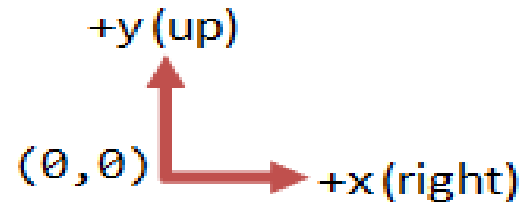
`glColor3f(0.0, 1.0, 1.0)` – cyan

`glColor3f(1.0, 1.0, 1.0)` – white

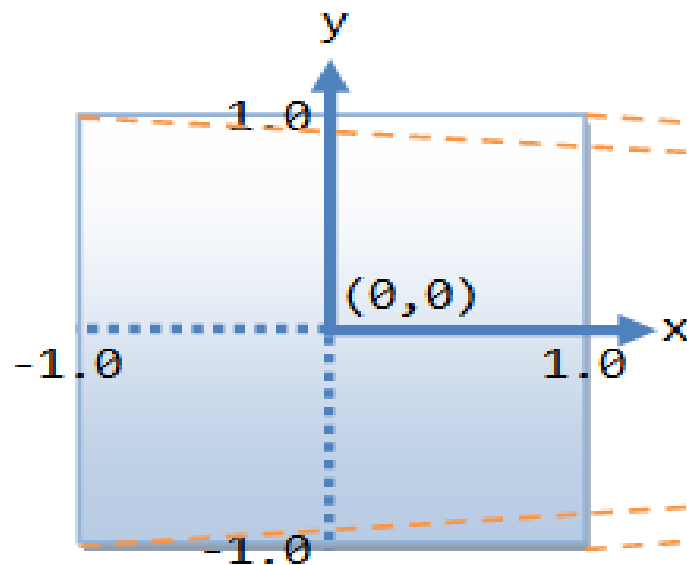
Co-ordinate System in OpenGL



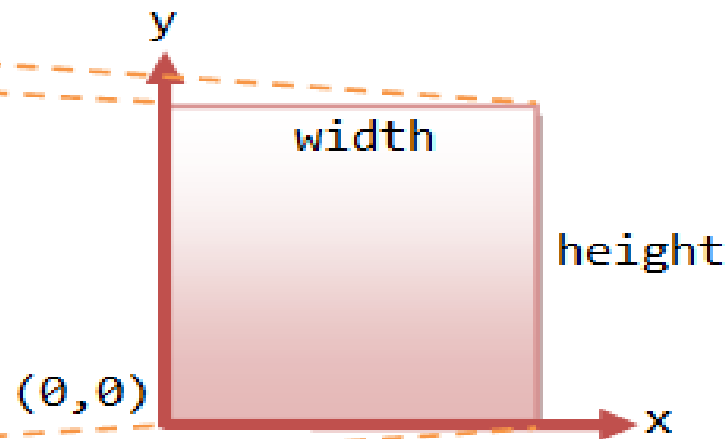
OpenGL 2D Coordinates



Viewport Coordinates



Clipping-Area
(default of 2x2 square
centered at origin)



Viewport
(width-by-height
in pixels)



GLUT

- We learnt OPENGGL state functions and primitive functions.
- We can't write complete program with these function only.
- These functions are not sufficient to generate graphics.
- OPENGGL does not have window related functions.
- So, we need to use GLUT for window related functions.



GLUT

- Stands for Graphics Library Utility Toolkit.
- It includes windows related functions such as defining window size, window position, creating window.
- It has function to initialize the graphics mode so that scene can be rendered.
- It also has function to call the scene rendering user defined callback function.
- To include this library just include `<GL/glut.h>`
- It will include `windows.h`, `gl.h` and `glu.h` automatically.
- However, glut does not contains functions to create sliders, menus and dialog boxes.
- For this, it will take help from windows based libraries.



GLUT

- Some of the important functions are
 - glutInit()
 - glutInitWindowPosition()
 - glutInitWindowSize()
 - glutInitDisplayMode()
 - glutCreateWindow()
 - glutDisplayFunc()
 - glutMainLoop()



glutInit(<argC>, <argV>)

- initializes graphics mode so that scene can be rendered.
- Takes two parameters from main function.
- glutInit(&argC, argV);
- The parameters are for command line argument.
- It must be called before any other glut routine.



glutInitWindowSize()

- `glutInitWindowSize(<width>,<height>)`
- It defines the window size to be created.
- the default size of (300, 300)
- This must be called before creating window.
- Takes two parameter width and height of window.

glutInitWindowPosition()

- `glutInitWindowPosition(<left-space>,<top-space>)`
- It defines the position of window to be created.
- The default position is (0,0) which might not be desirable.
- This must be called before creating window.
- Takes two parameter width and height
- Left space is gap between left margin of actual screen and the virtual screen.
- Top space is gap between top margin of actual screen and virtual screen.

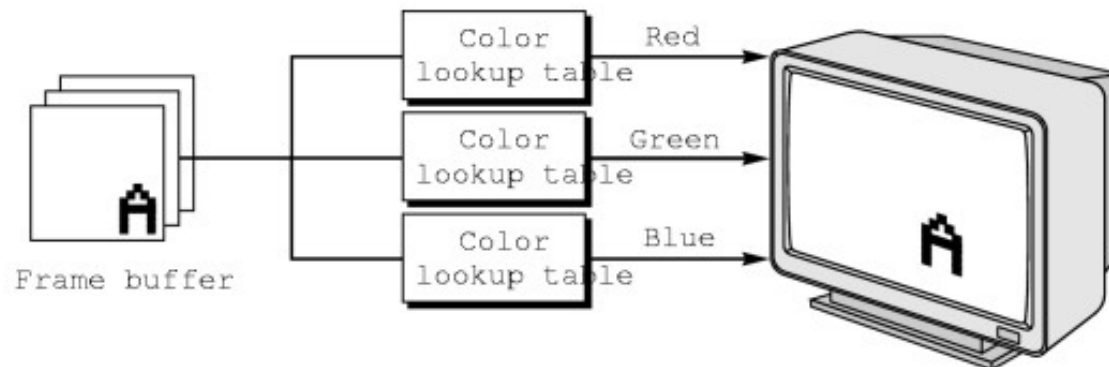
glutDisplayMode()

- `glutDisplayMode(<GLUT_RGBA> | <GLUT_RGB> | <GLUT_INDEX> | <GLUT_SINGLE> | <GLUT_DOUBLE> | <GLUT_DEPTH>)`
- Define different properties for frame buffer such as type of color to be used, number of buffers to be used and depth buffer.
- GLUT_RGBA selects colored frame buffer with transparency.
- It is default if neither GLUT_RGB nor GLUT_INDEX are specified.
- GLUT_RGB is an alias for GLUT_RGBA.
- GLUT_INDEX selects a color from indexed lookup table.
- It overrides GLUT_RGBA if specified.
- GLUT_SINGLE selects a single frame buffer.
- It is the default if GLUT_DOUBLE is not specified.
- This will help to synchronized speed between video controller and graphics processor.
- GLUT_DEPTH is used for detecting hidden objects or surfaces.

glutDisplayMode()

[Indexed Color]

- Colors are indices into tables of RGB values
- Requires less memory
 - indices usually 8 bits





glutCreateWindow()

- `glutCreateWindow(<title of window>);`
- Create window with given title.
- However, the window will be closed immediately if we do not call `glutMainLoop()` is called.
- This window is also called screen window.
- This where we draw every thing to simulate the graphics process.

glDisplayFunc()

- It calls user defined scene rendering callback function.
- It means its parameter is function name.
- The function generally draws on the screen window.
- `glutDisplayFunc(draw);`
- It calls draw function.

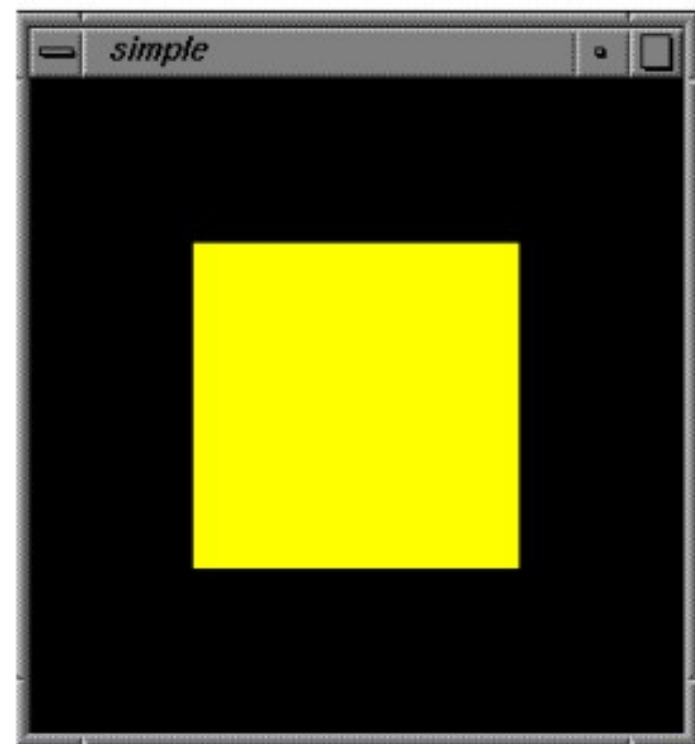


glMainLoop()

- It displays the graphics output window till we do not close the window.
- It must be called after calling glutDisplayFunc().
- Otherwise, it will throw exception.

Code Example

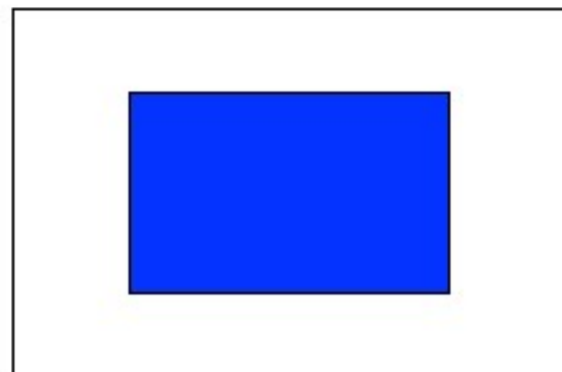
```
void Display()
{
    glClear(GL_COLOR_BUFFER_BIT);
    glColor4f(1,1,0,1);
    glBegin(GL_POLYGON);
        glVertex2f(-0.5, -0.5);
        glVertex2f(-0.5, 0.5);
        glVertex2f(0.5, 0.5);
        glVertex2f(0.5, -0.5);
    glEnd();
    glFlush();
}
....
```



Sample Example



```
Void DrawQuad(GLfloat color[])  
{  
    glColor3f(0,0,1);  
    glBegin(GL_QUADS);  
    glVertex2f(0,0);  
    glVertex2f(1.0, 0,0);  
    glVertex2f(1.0, 1.0);  
    glVertex2f(0.0, 1.0);  
    glEnd();  
}
```



- Most of window-based programs are **event-driven**
 - which means do nothing until an event happens, and then execute some pre-defined functions
- Events – key press, mouse button press and release, window resize, etc.



And many more ...

GLUT

- `glutKeyboardFunc()` – register the callback that will be called when a key is pressed
- `glutMouseFunc()` – register the callback that will be called when a mouse button is pressed
- `glutMotionFunc()` – register the callback that will be called when the mouse is in motion while a button is pressed
- `glutIdleFunc()` – register the callback that will be called when nothing is going on (no event)

Displaying points

```
#include<GL/glut.h>

void draw(){
    glClear(GL_COLOR_BUFFER_BIT);
    glPointSize(5);

    glBegin(GL_POINTS);
        glVertex2f(0.0, 0.0);
        glVertex2f(0.0, 0.6);
        glVertex2f(0.3, -0.3);
        glVertex2f(0.2, 0.8);
        glVertex2f(0.4, 0.0);
    glEnd();
    glFlush();
}
```

Displaying points

```
int main(int argc, char *argv[]){
    glutInit(&argc,argv);
    glutInitWindowSize(500,500);
    glutInitWindowPosition(200,200);
    glutInitDisplayMode(GLUT_RGB | GLUT_SINGLE);
    glutCreateWindow("First Glut");
    glClearColor(1,1,1,1);
    glColor3f(0.0,0.0,0.0);
    glutDisplayFunc(draw);
    glutMainLoop();
    return 0;
}
```

First Glut



Displaying Lines

```
void drawLines(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0f, 1.0f, 0.0f); // Green
    glBegin(GL_LINES);
        glVertex2i(50, 200); // first horizontal line
        glVertex2i(400, 200);
        glVertex2i(100, 300); // first vertical line
        glVertex2i(100, 100);
    glEnd();
    glFlush();
}
```

First Glut



Displaying Lines

```
void drawLines(){
    glClear(GL_COLOR_BUFFER_BIT);
    glColor3f(0.0f, 1.0f, 0.0f); // Green
    glBegin(GL_LINES);
        glVertex2i(0.1, 0.3); // first horizontal line
        glVertex2i(0.9, 0.3);
        glVertex2i(0.1, 0.1); // first vertical line
        glVertex2i(0.1, 0.9);
    glEnd();
    glFlush();
}
```

Displaying Lines

```
int main(int argc, char *argv[]){
    glutInit(&argc,argv);
    glutInitWindowSize(700,500);
    glutInitWindowPosition(200,200);
    glutInitDisplayMode(GLUT_RGB | GLUT_SINGLE);
    glutCreateWindow("First Glut");
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0,640,0,480);
    glClearColor(1,1,1,1);
    glColor3f(0.0,0.0,0.0);
    glutDisplayFunc(drawLines);
    glutMainLoop();
    return 0;
}
```



Input line and display it

```
#include<GL/glut.h>
#include<iostream>
using namespace std;
void drawLine(int x1,int y1,int x2,int y2){
    int x1,y1,x2,y2;
    cout<<"Enter first point"<<endl;
    cin>>x1>>y1;
    cout<<"Enter second point"<<endl;
    cin>>x2>>y2;
    glColor3f(0.0f, 1.0f, 0.0f); // Green
    glBegin(GL_LINES);
        glVertex2i(x1, y1); // first horizontal line
        glVertex2i(x2, y2);
    glEnd();
    glFlush();
}
```

Displaying Lines

```
int main(int argc, char *argv[]){
    glutInit(&argc,argv);
    glutInitWindowSize(700,500);
    glutInitWindowPosition(200,200);
    glutInitDisplayMode(GLUT_RGB | GLUT_SINGLE);
    glutCreateWindow("First Glut");
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0,640,0,480);
    glClearColor(1,1,1,1);
    glColor3f(0.0,0.0,0.0);
    glutDisplayFunc(drawLines);
    glutMainLoop();
    return 0;
}
```

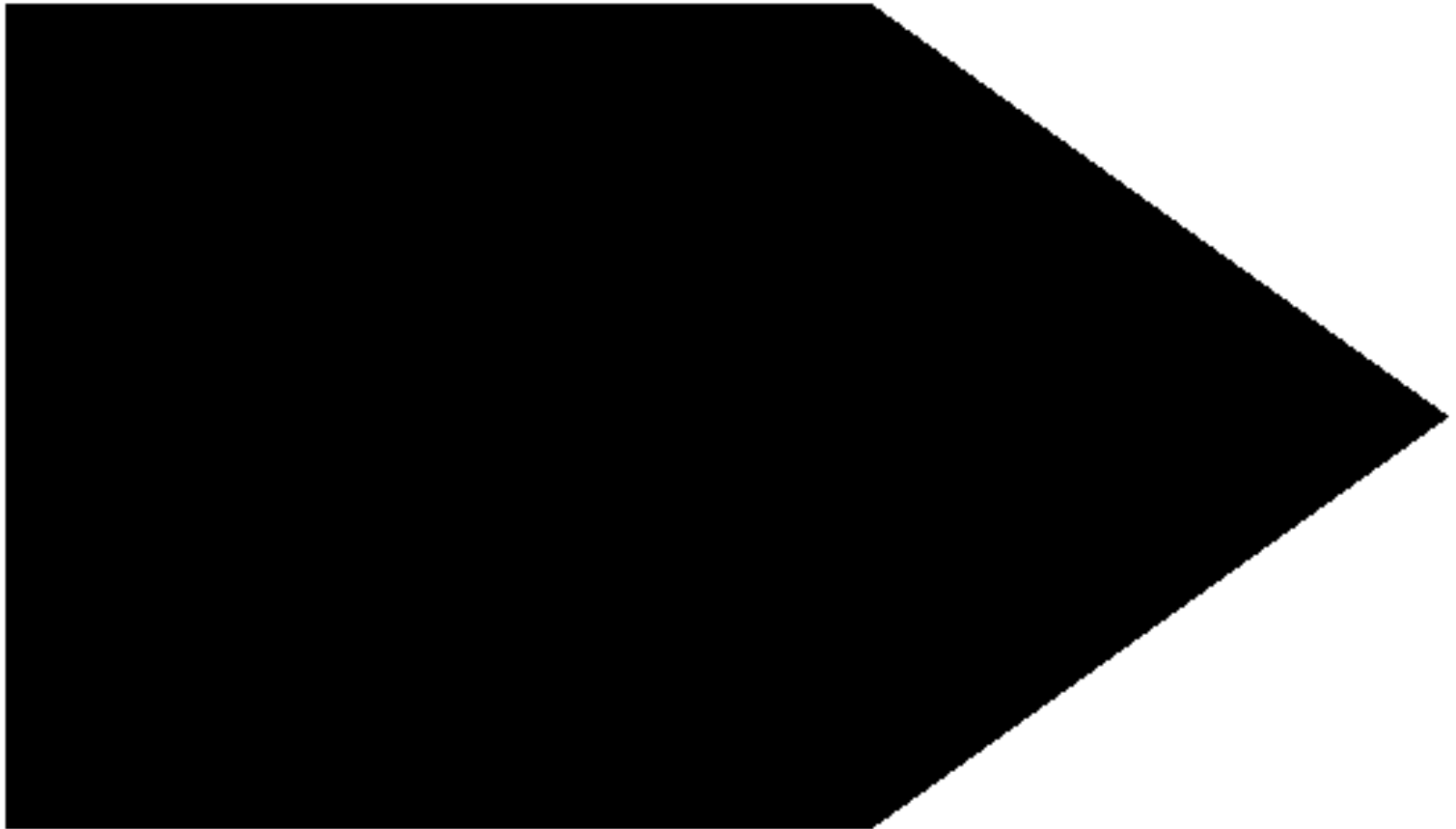


Displaying Polygon

```
void drawPolygon(){
    glClear(GL_COLOR_BUFFER_BIT);
    glPolygonMode(GL_FRONT, GL_FILL);
    glPolygonMode(GL_BACK, GL_LINE);
    glBegin(GL_POLYGON);
        glVertex2f(100, 100);
        glVertex2f(400, 100);
        glVertex2f(600, 250);
        glVertex2f(400, 400);
        glVertex2f(100, 400);
    glEnd();
    glFlush();
}
```

Displaying Polygons

```
int main(int argC, char *argV[]){
    glutInit(&argC,argV);
    glutInitWindowSize(800,800);
    glutInitWindowPosition(200,200);
    glutInitDisplayMode(GLUT_RGB | GLUT_SINGLE);
    glutCreateWindow("First Glut");
    glClearColor(1,1,1,1);
    glColor3f(0.0,0.0,0.0);
    glMatrixMode(GL_PROJECTION);
    glLoadIdentity();
    gluOrtho2D(0,740,0,780);
    glutDisplayFunc(drawPolygon);
    glutMainLoop();
    return 0;
}
```



Draw Constellation

```
int x,y,x_max=700,y_max=700;
void updateCoordinate(){
    x=rand() % x_max;
    y=rand()%y_max;
    glutPostRedisplay();
}
void drawContellation(){
    glPointSize(5);
    glColor3f(r,g,b);
    glBegin(GL_POINTS);
        glVertex2f(x, y);
        updateCoordinate();
    glEnd();
    glFlush();
}
```



Draw Constellation

```
int main(int argc, char *argv[]){  
    glutInit(&argc,argv);  
    myInit();  
    glutDisplayFunc(draw);  
    glutMainLoop();  
    return 0;  
}
```

Draw Constellation

